CG-N7 — Código Gênesis: O Legado de N-7

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	CG-N7 — Código Gênesis: O Legado de N-7
Gerente de Projeto	Izadora de Souza
Gerente/Responsável de GCS-DevOps	Gabriel Nader Marini
Fábrica de Software / Squad	IGJL Desenvolvimento
Repositório principal	https://github.com/GabNader/codigo-genesis-n7

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
23/09/2026	1.1	Adaptação do PGC para o projeto Código Gênesis: O Legado de N-7 (PAC Extensionista III).	Gabriel Nader Marini

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente.)

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado aos quatro integrantes da equipe do projeto de extensão universitária PAC Extensionista III, responsáveis pelo desenvolvimento do jogo Código Gênesis: O Legado de N-7 — um jogo 2D tático de aprendizagem de matemática construído na Godot Engine (versão 4.6.1). O plano abrange o repositório de código-fonte do jogo (cenas, scripts GDScript e demais assets) e os processos de versionamento, build e distribuição do executável, ainda em fase inicial de estruturação.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

Plano de Trabalho do PAC Extensionista III; documentação oficial da Godot Engine (docs.godotengine.org); repositório Git do projeto (a ser criado no GitHub).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas (ou planejadas) pelo projeto — a tabela foi ajustada em relação ao template, removendo categorias não aplicáveis a um jogo distribuído localmente (containers, IaC, monitoramento de ambientes em nuvem etc.):

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento do código-fonte (scripts GDScript) e dos assets do jogo (cenas, arte, áudio). Em uso.
Orquestração de CI/CD	GitHub Actions	Automação do build/export do projeto Godot a cada Pull Request. Planejado — ainda não implementado.
Empacotamento	Godot Engine (export	Geração do build/executável versionado do jogo para a

Categoria	Ferramenta(s) sugerida(s)	Finalidade
	templates)	plataforma alvo (ex.: Windows). Em uso manual pela equipe; automação via CI/CD é planejada.
Rastreamento de tarefas	GitHub Issues / Projects	Vinculação de commits/PRs a itens de trabalho. Planejado.

3.2 Ambientes e Infraestrutura

Como o projeto não envolve infraestrutura em nuvem (ver observação abaixo), os "ambientes" listados a seguir representam estágios do fluxo de trabalho da equipe, não servidores provisionados.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Ambiente local de cada integrante da equipe (editor da Godot Engine), onde o código e os assets são criados e testados antes do commit.	Commit/push na branch de feature
Build de Teste (equipe)	Build executável exportado pela Godot e compartilhado entre a equipe, para testar a jogabilidade e validar as mecânicas antes da entrega/apresentação.	Merge na branch principal
Entrega/Apresentação	Build final exportado, utilizado para a apresentação e verificação do PGC/repositório em laboratório, e disponibilizado para download pelos avaliadores.	Validação do build de teste pela equipe e aprovação da Gerente de Projeto

Não há, no momento, ambientes hospedados em nuvem: o jogo é executado localmente, a partir de um build exportado pelo próprio editor da Godot Engine, sem servidores, DNS ou banco de dados dedicados. Credenciais (ex.: token de acesso do GitHub) devem ser mantidas apenas localmente por cada integrante, nunca versionadas no repositório.

3.3 Estratégia de Branching e Versionamento

A equipe ainda não formalizou uma estratégia de branching. Propõe-se a adoção de uma estratégia simplificada de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real assim que o repositório for criado:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Dockerfiles e manifests de containers (não aplicável a este projeto)
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes) — não aplicável a este projeto
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.
- Scripts de migração de banco de dados (N/A)
- Documentação técnica (README, ADRs — Architecture Decision Records)
- Assets do jogo — cenas (.tscn), arte, áudio e fontes — versionados junto ao código-fonte no mesmo repositório

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<ticket>-<descrição-curta>	feature/SIQQ-123-login-oauth
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(auth): adiciona login via OAuth2
Tag de release	Versionamento Semântico vX.Y.Z	v2.3.0
Build do jogo (executável)	CodigoGenesis-N7_<plataforma>_v<versão>	CodigoGenesis-N7_Windows_v0.3.0
...	...	...

4.3 Estrutura do Repositório

Estrutura planejada para o repositório: /scenes (cenas .tscn do jogo), /scripts (código GDScript), /assets (arte, áudio e fontes), /addons (plugins da Godot Asset Library, se houver), project.godot e export_presets.cfg na raiz, /docs (documentação do projeto) e, quando criado, /.github/workflows (definições de pipeline de CI).

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (com cache).
2. Build da aplicação.
3. Análise estática de código (lint) e verificação de padrões de codificação.
4. Testes unitários, com medição de cobertura mínima.

5. Testes de integração.
6. Análise de segurança do código (SAST) e de dependências (SCA).
7. Empacotamento e publicação do artefato/imagem versionado no registro.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro desenvolvedor/colega antes do merge ("quatro olhos").
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate).

5.3 Critérios de Qualidade (Quality Gate)

Critérios adotados: o projeto abre sem erros nem warnings críticos no editor da Godot; o build de exportação é gerado com sucesso para a plataforma alvo; e o Pull Request recebe a aprovação de ao menos 1 outro integrante da equipe antes do merge. Testes automatizados ainda não fazem parte do fluxo, dado o estágio atual do projeto.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção. Como o Código Gênesis não é implantado em servidor (é distribuído como executável local), os conceitos abaixo são adaptados: "ambientes" correspondem aos estágios definidos na seção 3.2 (Desenvolvimento → Build de Teste → Entrega/Apresentação) e "produção" corresponde ao build final entregue para avaliação.

6.1 Etapas do Pipeline de CD

8. Deploy automático em ambiente de Homologação/Staging.
9. Execução de testes automatizados de aceitação/E2E em staging.
10. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
11. Deploy em produção com estratégia definida (ver 6.2).
12. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
13. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. O pipeline de CD deve manter a versão anterior disponível (imagem no registro, IaC versionada) para permitir rollback automatizado em caso de falha nos testes de fumaça ou nos indicadores de saúde pós-deploy.

7. Infraestrutura como Código (IaC)

Este item não é aplicável ao Código Gênesis: o projeto não provisiona servidores, containers ou serviços de nuvem — o jogo roda localmente a partir de um executável exportado pela Godot Engine. Não há, portanto, infraestrutura a ser descrita como código. A seção é mantida no plano apenas por completude em relação ao template.

7.1 Versionamento de Infraestrutura

- Alterações de infraestrutura seguem o mesmo processo de PR + revisão + pipeline ("plan" automático em PR, "apply" após aprovação).
- O estado da infraestrutura (state file) é armazenado de forma centralizada e versionada, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Sempre que viável, ambientes de revisão (review apps) devem ser provisionados automaticamente por Pull Request, para validação isolada, e destruídos automaticamente após o merge ou fechamento do PR.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

O CCM deste projeto é composto pela Gerente de Projeto (Izadora de Souza) e pelo Responsável de GCS-DevOps (Gabriel Nader Marini). É acionado apenas em mudanças que alterem o escopo pedagógico combinado com o orientador do PAC ou a arquitetura central do projeto (ex.: reestruturação de cenas, troca de mecânica principal do jogo). Mudanças de rotina seguem apenas o fluxo de Pull Request.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no registro e ao estado da infraestrutura correspondente.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção devem ser monitorados continuamente (métricas, logs e alertas), permitindo identificar rapidamente falhas decorrentes de uma implantação.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

Marcos previstos para a evolução da GCS/DevOps do projeto: (1) criação e publicação do repositório Git no GitHub; (2) definição e adoção da estratégia de branching (main + feature branches); (3) implantação de um pipeline de CI via GitHub Actions para validar o build do projeto a cada Pull Request; (4) primeira geração automatizada de build/release versionado (tag SemVer); (5) apresentação/entrega final do PAC Extensionista III com o build de referência do jogo.

12. Treinamento e Recursos

Além da Godot Engine (já em uso pela equipe), a adoção deste plano exige que os quatro integrantes se familiarizem com o Git (comandos básicos de branch, commit e Pull Request) e, futuramente, com a sintaxe de workflows do GitHub Actions. Como recursos, serão usados a documentação oficial do GitHub Actions, a

documentação da Godot Engine sobre exportação de builds, e a orientação do professor responsável pelo PAC Extensionista III.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

Este item se aplica apenas a eventuais assets de terceiros obtidos na Godot Asset Library ou em bancos de arte/áudio livres, que devem ser versionados junto ao repositório com o devido registro de licença de uso. O projeto não possui subcontratados ou fornecedores externos de software.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de segurança, publicação do artefato/imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Staging	Deploy automático em homologação + testes de aceitação.	Merge na branch principal
Gate de Release	Aprovação para produção (automática ou manual, conforme criticidade).	Sucesso dos testes em staging
CD — Produção	Deploy (rolling/blue-green/canary), smoke tests, geração de baseline/tag.	Aprovação do gate de release

Nota: *este template deve ser adaptado ao contexto real do projeto (ferramentas, criticidade do sistema e maturidade da equipe em DevOps), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação.*